ΘΕΜΑ Α

Α1. Χαρακτηρισμός προτάσεων Σωστό / Λάθος

- **α. Σωστό**: Η ουρά (Queue) ακολουθεί τη δομή FIFO (First In First Out - Πρώτο μέσα, Πρώτο έξω).
- **β. Λάθος:** Η έκφραση slicing word[a:b] επιστρέφει το τμήμα από το word[a] μέχρι και το στοιχείο word[b-1]. Το στοιχείο στη θέση b δεν συμπεριλαμβάνεται.
- **γ. Σωστό**: Οι τοπικές μεταβλητές (local variables) έχουν εμβέλεια (scope) μόνο μέσα στο υποπρόγραμμα (συνάρτηση) στο οποίο έχουν δηλωθεί.
- **δ. Λάθος:** Η κλάση είναι απλώς ένα πρότυπο (blueprint). Για να υπάρξει ένα αντικείμενο, πρέπει να γίνει ρητή δημιουργία του (στιγμιότυπο - instantiation).
- **ε. Σωστό**: Όταν ο τελεστής + εφαρμόζεται σε αλφαριθμητικά (strings), κάνει συνένωση (concatenation).

Α2. Αντιστοίχιση

1. pow(4,2) υπολογίζει το 4 στο τετράγωνο (4^2 = 16). Άρα: **1 - γ**
2. divmod(2,5) επιστρέφει το πηλίκο και το υπόλοιπο της ακέραιας διαίρεσης (2 // 5 = 0 και 2 % 5 = 2). Άρα: **2 - β**
3. (5<2) or (4>=3) αξιολογείται ως False or True, το οποίο δίνει True. Άρα: **3 - α**
4. not(7!=4+5) and (5>3) αξιολογείται ως not(7!=9) and True -> not(True) and True False and True, το οποίο δίνει False. Άρα: **4 - ε**
5. len('Python') επιστρέφει το πλήθος των χαρακτήρων της λέξης, που είναι 6.
   Άρα: **5 - στ**

(Το γράμμα δ (2,0) περισσεύει)

ΘΕΜΑ Β

Β1. Συμπλήρωση Κενών Ο κώδικας ελέγχει αν μια λίστα είναι ταξινομημένη σε αύξουσα σειρά. Για να γίνει αυτό, συγκρίνει κάθε στοιχείο με το προηγούμενό του. Αν έστω και ένα στοιχείο είναι μικρότερο από το προηγούμενο, η λίστα δεν είναι ταξινομημένη.

- (1) **len(A)** : Υπολογίζει το πλήθος των στοιχείων της λίστας Α.
- (2) **N** (ή εναλλακτικά len(A)) : Το εύρος της επανάληψης range(1, N).
- (3) **A[j-1]** : Ελέγχει αν το τρέχον στοιχείο A[j] είναι μικρότερο από το *προηγούμενο* A[j-1].
- (4) **False** : Αν βρεθεί στοιχείο μικρότερο από το προηγούμενο, η ταξινόμηση δεν ισχύει, άρα η μεταβλητή-σημαία f γίνεται False.

- (5) **f** : Επιστρέφει τη λογική μεταβλητή f (η οποία έχει μείνει True αν δεν βρέθηκε κανένα λάθος, ή έγινε False).

Β2. Εκτέλεση τμήματος προγράμματος με Λίστες

- **(1) ['MANOLIS', 'MARIA', 'NIKOS', 'LAZAROS']** *(Προστέθηκε ο 'MANOLIS' στην αρχή της λίστας).*
- **(2) ['MANOLIS', 'MARIA', 'NIKOS']** *(Η συνάρτηση pop() χωρίς όρισμα διαγράφει το τελευταίο στοιχείο, τον 'LAZAROS').*
- **(3) ['MANOLIS', 'MARIA', 'ADAM', 'NIKOS']** *(Η insert(2, 'ADAM') εισάγει τον 'ADAM' στη θέση με δείκτη 2).*
- **(4) ['MANOLIS', 'ADAM', 'NIKOS']** *(Η pop(1) διαγράφει το στοιχείο στη θέση 1, δηλαδή τη 'MARIA').*
- **(5) ['MANOLIS', 'ADAM', 'NIKOS', 'ANNA']** *(Προστέθηκε η 'ANNA' στο τέλος της λίστας με συνένωση).*

Β3. Μετατροπή while σε for

Python

```
for i in range(1, 4):          # Το i παίρνει τιμές από το 1 έως το 3 (το 4 δεν συμπεριλαμβάνεται)
    for j in range(5, 0, -1): # Το j παίρνει τιμές από το 5 έως το 1, μειούμενο κατά 1 (-1 βήμα)
        print i, j             # Τυπώνει τις τιμές των i και j
```

ΘΕΜΑ Γ

Αυτό το πρόβλημα απαιτεί επαναληπτική διαδικασία που τερματίζει με τιμή ('ΤΕΛΟΣ'), εύρεση μέσου όρου και διαχείριση αρχείων.

```
# Άνοιγμα αρχείου για εγγραφή των ονομάτων με βαθμολογία > 7
f = open('results.txt', 'w')

pl = 0  # Μετρητής για το συνολικό πλήθος των διαγωνιζόμενων (χρησιμοποιείται στο Γ4)
k = 0   # Μετρητής για όσους έχουν τελική βαθμολογία <= 7 (χρησιμοποιείται στο Γ4)

# Ερώτημα Γ1: Πρώτη εισαγωγή ονόματος εκτός της δομής επανάληψης
name = raw_input('Δώσε όνομα: ')

# Η επανάληψη τερματίζει όταν δοθεί ως όνομα η λέξη 'ΤΕΛΟΣ'
while name != 'ΤΕΛΟΣ':
    s = 0  # Αρχικοποίηση του αθροιστή βαθμών για τον τρέχοντα μαθητή

    # Επανάληψη 5 φορές για τους 5 κριτές
    for i in range(5):

        vathmos = int(raw_input('Δώσε βαθμολογία: ')) # Χρήση int(raw_input()) που είναι η πιο ασφαλής συντακτικά μέθοδος στην Python 2.7

        s = s + vathmos  # Προσθήκη του βαθμού στο άθροισμα
```

```
# Ερώτημα Γ2: Υπολογισμός και εμφάνιση του Μέσου Όρου
    mo = float(s) / 5  # Μετατροπή του s σε float για να γίνει πραγματική διαίρεση
    print mo

    # Ερώτημα Γ3: Έλεγχος αν ο Μ.Ο. είναι πάνω από 7 για εγγραφή στο αρχείο
    if mo > 7:
        f.write(name + '\n')  # Εγγραφή του ονόματος και αλλαγή γραμμής στο αρχείο
    else:
        # Ερώτημα Γ4: Αν είναι <= 7, αυξάνεται ο μετρητής k
        k = k + 1

    pl = pl + 1  # Αυξάνεται το συνολικό πλήθος των διαγωνιζόμενων

    # Επανεισαγωγή ονόματος στο τέλος του βρόχου while
    name = raw_input('Δώσε όνομα: ')

# Ερώτημα Γ4: Υπολογισμός και εμφάνιση ποσοστού μετά το τέλος της while
pososto = (float(k) / pl) * 100
print pososto

# Κλείσιμο του αρχείου
f.close()
```

ΘΕΜΑ Δ

Αυτό το πρόβλημα συνδυάζει μονοδιάστατες λίστες, συναρτήσεις για εύρεση μέγιστου και παράλληλη ταξινόμηση (Bubble Sort).

```
# Συνάρτηση για το ερώτημα Δ3
# Δέχεται τις δύο λίστες και επιστρέφει απευθείας το όνομα του βιβλίου με τις μέγιστες πωλήσεις
def MEGISTOS(SALES, TITLES):
    max_pol = SALES[0]   # Θεωρούμε ως μέγιστες πωλήσεις τις πωλήσεις του 1ου βιβλίου
    max_on = TITLES[0]    # Θεωρούμε ως αντίστοιχο τίτλο τον τίτλο του 1ου βιβλίου

    # Σύγκριση με τα υπόλοιπα 39 στοιχεία των λιστών
    for i in range(1, len(SALES)):
        if SALES[i] > max_pol:
            max_pol = SALES[i]  # Ενημέρωση των μέγιστων πωλήσεων
            max_on = TITLES[i]   # Ενημέρωση του αντίστοιχου τίτλου

    return max_on  # Επιστροφή του τίτλου

# ----------------- ΚΥΡΙΩΣ ΠΡΟΓΡΑΜΜΑ -----------------

TITLES = []
SALES = []

# Ερώτημα Δ1: Γέμισμα λιστών για 40 βιβλία με έλεγχο εγκυρότητας
for i in range(40):
    title = raw_input('Δώσε τίτλο βιβλίου: ')
    TITLES.append(title)

    poliseis = int(raw_input('Δώσε αριθμό πωλήσεων: '))
    # Έλεγχος εγκυρότητας: οι πωλήσεις πρέπει να είναι μη αρνητικός αριθμός (>= 0)
    while poliseis < 0:
        poliseis = int(raw_input('Δώσε ξανά αριθμό πωλήσεων γιατί δεν ήταν σωστός: '))
    SALES.append(poliseis)

# Ερώτημα Δ2: Υπολογισμός και εμφάνιση αθροίσματος και μέσου όρου πωλήσεων
```

```
athr = 0
for i in range(40):
    athr = athr + SALES[i]

mo = float(athr) / 40
print athr
print mo

# Ερώτημα Δ3: Κλήση της συνάρτησης MEGISTOS και εμφάνιση του αποτελέσματος
max_tit = MEGISTOS(SALES, TITLES)
print 'Ο τίτλος του βιβλίου με τις περισσότερες πωλήσεις είναι: ', max_tit

# Ερώτημα Δ4: Ταξινόμηση Φυσαλίδας (Bubble Sort) με τα ακριβή όρια του σχολικού βιβλίου
N = 40
for i in range(N-1):
    for j in range(N-1, i, -1):
        # Σύγκριση για αλφαβητική (αύξουσα) ταξινόμηση με βάση τη λίστα TITLES
        if TITLES[j] < TITLES[j-1]:
            # Ταυτόχρονη αντιμετάθεση (swap) στη λίστα TITLES
            temp = TITLES[j]
            TITLES[j] = TITLES[j-1]
            TITLES[j-1] = temp

            # Παράλληλη αντιμετάθεση (swap) στη λίστα SALES για διατήρηση της
αντιστοιχίας
            temp2 = SALES[j]
            SALES[j] = SALES[j-1]
            SALES[j-1] = temp2

# Ερώτημα Δ4 (συνέχεια): Εμφάνιση των στοιχείων των λιστών ανά ζεύγη (γραμμή προς γραμμή)
for i in range(40):
    print TITLES[i], SALES[i]
```